

1 RideFlow Complete Codebase and Maintenance Guide

Repository: [raiven427/ride-flow](https://github.com/raiven427/ride-flow)

Current deployment handoff: private GitHub main branch, commit 1eeab2f

Current stack: React, TypeScript, Vite, Express, tRPC, Drizzle ORM, MySQL/TiDB-compatible SQL, S3-compatible storage, provider-neutral OIDC, Stripe Connect scaffolding, and M-Pesa Daraja scaffolding.

This guide explains where the code lives, how the pieces work together, and how to update RideFlow safely. It is an operator and self-study guide, not a substitute for professional legal, tax, security, or payment advice.

1.1 1. What RideFlow is

RideFlow is a ride-hailing marketplace prototype with customer and driver experiences, transparent KSh fare quotes, database-backed Nairobi fare rules, a 5% platform commission ledger, driver onboarding documents, profile storage, admin controls, and self-hosting adapters. The user interface is intentionally polished, but a real transport marketplace also requires dispatch, maps, realtime location, production payment reconciliation, support, fraud controls, and safety operations.

The current code is prepared so that your own server can run the application. Credentials are configuration, not source code. The repository contains schema and migration blueprints, but it does not contain your production database data or secret values.

1.2 2. Repository map

Path	Responsibility	Change it when
client/src/App.tsx	React application shell, providers, and routing	You add or reorganize top-level screens
client/src/pages/Home.tsx	Main RideFlow customer/driver workspace and interactive demo flows	You change booking, onboarding, admin, profile, or dashboard UI
client/src/index.css	Global visual system, responsive layout, and motion	You change colors, typography, spacing, responsive behavior, or animations
client/src/lib/trpc.ts	Typed browser-to-server tRPC binding	Rarely; update only when changing the API transport
client/src/const.ts	Browser-side OIDC authorization configuration	You change the identity provider authorization endpoint
client/src/components/	Reusable UI and template components	You add a reusable interaction or layout primitive

Path	Responsibility	Change it when
server/_core/index.ts	Express entry point, health check, webhook routes, tRPC, Vite/static serving	You change server startup, webhooks, HTTP middleware, or deployment behavior
server/routers.ts	Typed API contract for profile, admin, fare, file, and payment procedures	You add or change a frontend-callable backend action
server/db.ts	Drizzle database access helpers	You add queries, persistence, or transaction behavior
server/fare.ts	Pure server-side KSh fare calculation	You change fare formulas or commission rules
server/payments.ts	Stripe and Daraja configuration, readiness, PaymentIntent, token, and STK Push helpers	You change provider payment behavior
server/storage.ts	S3-compatible private object storage adapter	You change upload/download provider configuration
server/_core/env.ts	Server-side environment contract	You add or rename deployment variables
server/_core/oauth.ts and server/_core/sdk.ts	Provider-neutral OIDC callback and session boundary	You change login provider behavior
server/_core/notification.ts	Notification webhook adapter	You change signup or operations notifications
drizzle/schema.ts	Database table definitions and types	You add or change persistent data
drizzle/*.sql	Reviewed migration files	You apply a schema change to a database
server/*.test.ts	Unit and authorization regression tests	You add behavior or fix a bug
docs/	Operations, deployment, payment, and database guides	You change setup assumptions or support procedures
package.json	Dependencies and scripts	You add packages or commands
docs/deployment-environment.template	Complete environment variable checklist with placeholders	You add a configuration key; never add real values

1.3 3. How a request moves through the app

The browser renders React components from `client/src`. A page calls a typed tRPC hook. The tRPC client sends the request to `/api/trpc`. Express creates a request context, resolves the logged-in user from the OIDC session, and routes the call to `server/routers.ts`. The procedure checks whether the action is public, protected, or admin-only. It then calls a database, storage, fare, notification, or payment helper.

A fare quote follows this sequence: the browser submits origin, destination, distance, and duration; the protected quote procedure reads active Nairobi rules; `server/fare.ts` calculates the fare and 5% commission on the server; `server/db.ts` persists the quote and ledger entries; the browser receives the transparent KSh breakdown. The browser is never the authority for the final money calculation.

A file upload follows a similar sequence: the authenticated browser sends validated file metadata and bytes; the protected procedure validates MIME type and size; `server/storage.ts` uploads bytes to a private S3-compatible bucket; `server/db.ts` stores metadata and the storage key; a short-lived signed URL can later be used for authorized access.

1.4 4. Database model

RideFlow uses Drizzle schema definitions in `drizzle/schema.ts`. The important groups are identity, profile, driver onboarding, files, fare rules, quotes, ledger entries, and admin settings.

Data group	Purpose	Important safety rule
Users	OIDC-backed identity and role	Never trust a client-supplied role
Profiles	Customer/driver profile and onboarding status	Protect personal and document data
Files	S3 key plus metadata and review status	Never store file bytes in SQL
Fare rules	Active city-specific price configuration	Update through admin authorization
Fare quotes	Expiring server-calculated fare snapshot	Recalculate final amounts server-side
Ledger entries	Rider charge, driver earning, platform commission	Treat as append-only financial events
Admin settings	Owner email, notification recipient, transferable ownership	Require admin authorization and audit changes

To add a table, edit `drizzle/schema.ts`, run `pnpm drizzle-kit generate`, inspect the generated SQL, and apply it through your approved database migration process. Never casually drop tables or delete rows in a production database. Add a backup and restore test before a destructive migration.

1.5 5. Fare and commission logic

The formula lives in `server/fare.ts`; the active configurable inputs live in the fare-rules table. The quote records base fare, distance fare, time fare, safety fee, subtotal, platform commission, rider total, and driver earnings. The current commission basis is 500 basis points, or 5%.

The formula must remain server-side. If you change the price per kilometre, minimum fare, safety fee, or commission, update the database-backed fare rule through the admin procedure or a reviewed migration. Then run `pnpm test`, inspect a quote in KSh, and verify that rider total, driver earnings, and platform commission reconcile.

The current app-level ledger intentionally exposes no update/delete procedures. Database-level immutability requires a database-engine-specific policy or trigger and should be reviewed separately for your chosen MySQL/TiDB provider.

1.6 6. Admin controls

The admin system is database-backed and initialized from the configured owner email. Admin procedures can read and update fare rules, update the notification email, view payment readiness, and transfer ownership to a verified user. The admin UI is part of `client/src/pages/Home.tsx`, while authorization lives in server procedures.

To change the admin, sign in as the current admin, enter the verified replacement user's email in the Admin Control Room, and confirm the transfer. Do not edit the frontend to create an admin. If you are locked out, use a controlled database recovery procedure and record the change.

1.7 7. Authentication and authorization

RideFlow uses provider-neutral OIDC variables. `client/src/const.ts` starts authorization. `server/_core/oauth.ts` receives the callback. `server/_core/sdk.ts` exchanges and reads provider tokens. The server session is signed using `JWT_SECRET`.

Use protected procedures for customer/driver actions and admin procedures for ownership, fares, notification settings, and payment status. The frontend may hide controls, but only the server can enforce access. Add an authorization test for every new privileged operation.

1.8 8. File storage

`server/storage.ts` uses standard S3-compatible configuration. The bucket should be private. Store the bucket key and metadata in SQL; provide users with short-lived signed URLs. Configure file size limits, MIME validation, retention, deletion, and incident response. Never commit access keys, signed URLs, identity documents, or storage dumps.

1.9 9. Stripe and M-Pesa Daraja

`server/payments.ts` reports whether each provider is enabled and configured. Stripe scaffolding creates server-side `PaymentIntents` and supports application-fee metadata for a marketplace flow. The Stripe webhook route verifies the raw body and signature. Daraja scaffolding requests a server-side access token, prepares STK Push requests, and accepts callback routes.

These helpers are not a substitute for provider onboarding or reconciliation. You still need Stripe Connect connected-account onboarding, refunds, disputes, payout handling, and event idempotency. You also need

Daraja production approval, callback testing, reversals, and a separate driver-disbursement design. Keep both feature flags false until sandbox tests are complete.

1.10 10. Environment configuration

Use docs/deployment-environment.template as the variable list. Put actual values in your hosting provider's secret manager. The most important groups are database, OIDC, storage, notifications, Stripe, and Daraja.

Variable group	Examples	Never expose
Database	DATABASE_URL	Database password in frontend or GitHub
Session	JWT_SECRET	Session signing secret
OIDC	0AUTH_CLIENT_SECRET	Provider client secret
Storage	S3_SECRET_ACCESS_KEY	Storage secret key
Stripe	STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET	Secret key and webhook signing secret
Daraja	Consumer secret, passkey, security credential	All Daraja credentials

Use test/sandbox values first. Rotate any key that appears in chat, screenshots, logs, or a public repository. Never commit a populated .env file.

1.11 11. Running on your PC

Install Node.js LTS, Git, pnpm, and a MySQL/TiDB-compatible database client. Clone the private repository, install dependencies, copy the environment template into your local secret manager or .env, and run:

```
git clone https://github.com/raiven427/ride-flow.git
cd ride-flow
pnpm install
pnpm check
pnpm test
pnpm dev
```

Use pnpm build to create a production build and pnpm start to run it. Use curl http://localhost:3000/healthz to verify the server process. Never use production payment keys on a laptop while experimenting.

1.12 12. Safe update workflow

Before changing code, create a branch with a meaningful name, such as feature/driver-documents or fix/daraja-callback. Make one focused change, add or update a Vitest test, run pnpm check, pnpm test, and pnpm build, then inspect the UI in a private preview.

Commit with a descriptive message and push the branch. Merge only after review. Keep main deployable. Before a schema change, back up the database and review the generated SQL. Before a payment change, use

sandbox credentials and provider test events. Before a deployment change, verify environment variables and webhook URLs.

A typical update looks like:

```
git checkout -b feature/my-change
# edit code and tests
pnpm check
pnpm test
pnpm build
git add -A
git commit -m "Describe the change"
git push -u origin feature/my-change
```

1.13 13. Deployment checklist

Check	Evidence to keep
HTTPS and domain	Certificate and DNS record
Health endpoint	Successful /healthz response
Database	Migration log, backup, restore test
Storage	Private bucket and signed URL test
OIDC	Successful login and callback
Stripe	Sandbox PaymentIntent and verified webhook
Daraja	Sandbox STK Push and callback result
Ledger	Quote totals reconcile with commission and driver earnings
Admin	Transfer and revoke test
Security	Secret scan, dependency audit, MFA, rate-limit test
Operations	Support contact, incident response, refund workflow

1.14 14. What is not finished by source code alone

The repository cannot create a compliant transport business by itself. Before real riders pay, you need driver verification operations, maps and routing, dispatch, realtime location, chat, scheduled-ride jobs, refunds, payout reconciliation, fraud controls, customer support, tax handling, terms, privacy policy, safety policies, and a tested incident response plan.

The payment scaffolding intentionally stops short of claiming that money movement is live. Provider accounts must be verified, webhooks must be registered, callbacks must be reachable, and every provider event must reconcile to the RideFlow ledger before a pilot.

1.15 15. Recommended study order

Start with HTML/CSS/JavaScript and Git. Then study TypeScript and React components, followed by HTTP and REST concepts. Next learn Node.js and Express, SQL and relational modeling, Drizzle migrations, authentication and sessions, object storage, webhooks, testing, Linux process management, HTTPS, and observability. Finally study marketplace payments, payout reconciliation, data protection, and transport operations.

A useful first project is to add a small admin-only fare rule field, write its test, run the migration in a disposable database, and review the result in the UI. This teaches the complete RideFlow loop from frontend input to authenticated procedure, database write, test, and deployment.

1.16 References

[1]: <https://docs.stripe.com/connect> Stripe Connect documentation [2]: <https://docs.stripe.com/connect/webhooks> Stripe Connect webhook documentation [3]: <https://docs.stripe.com/webhooks> Stripe webhook documentation [4]: <https://developer.safaricom.co.ke/> Safaricom Daraja developer portal [5]: <https://orm.drizzle.team/docs/migrations> Drizzle migrations documentation [6]: https://openid.net/specs/openid-connect-core-1_0.html OpenID Connect Core specification